

**A Report**  
**ON**  
**Digital Queue Management System**  
**BY**

Name of the Student

Shaik Asif  
T.Dheeraj Reddy  
K.Jaya Sri  
L.Hema Chandra  
T.Raja Sekhar Reddy

Registration No.

AP24110011468  
AP24110011473  
AP24110011478  
AP24110011516  
AP24110011528

**Prepared in the partial fulfillment of the**  
Project Based Learning of Course CSE 211: Full Stack Development

**SRM UNIVERSITY, AP**



**MAY 2026**

## INTRODUCTION

In today's fast-paced digital world, efficient customer service and time management have become critical for organizations across various sectors such as banking, healthcare, retail, and public services. Traditional queue management systems, which rely heavily on manual processes and physical waiting lines, often lead to overcrowding, long waiting times, and poor customer experience. These limitations highlight the need for a smarter and more efficient approach to managing queues.

The **Digital Queue Management System** is designed to address these challenges by providing a modern, technology-driven solution for handling customer queues. It is a web-based application that enables customers to join queues remotely, obtain digital tokens, and track their position in real time. By eliminating the need for physical waiting, the system enhances convenience, reduces congestion, and improves overall service efficiency.

This system is developed using the MERN stack, which includes MongoDB, Express.js, React.js, and Node.js. It integrates frontend user interaction with robust backend processing and a scalable database structure. Customers can easily access the system through mobile devices or kiosks, while administrators and service agents can manage queues, assign service counters, and monitor operations through a dedicated dashboard.

Additionally, the system provides features such as estimated waiting time, real-time updates, and notification alerts via SMS or email. These functionalities ensure better communication between the system and users, leading to increased transparency and customer satisfaction.

Overall, the Digital Queue Management System represents a significant improvement over traditional queue handling methods by leveraging modern web technologies to deliver an efficient, scalable, and user-friendly solution.

Additionally, the system incorporates QR code-based access, allowing users to quickly enter the queue system by simply scanning a code at the service location. This enhances accessibility and enables a completely contactless queue registration process.

## Scenario-Based Case Study

Meet **Rahul Verma**, a software engineer with a busy and tightly scheduled lifestyle who values efficiency, time management, and convenience in his daily activities. Rahul frequently visits banks to perform essential transactions such as cash deposits, account updates, and document verification. However, he often faces challenges such as long waiting queues, overcrowded service areas, and uncertainty about how long he must wait before being served. These issues not only waste his valuable time but also cause frustration and reduce his overall experience.

One day, Rahul visits a modern bank branch that has implemented a **Digital Queue Management System**, a smart web-based platform designed to streamline customer flow and eliminate traditional waiting line problems. Curious about this new system, Rahul decides to use it.

### User Access and Registration

As Rahul enters the bank, he notices a QR code displayed at the entrance along with a kiosk. He scans the QR code using his smartphone and is redirected to the queue management web application. The interface is simple and user-friendly. Rahul enters his basic details such as name and selects the service he requires (cash deposit). Within seconds, he is successfully registered in the system.

### Digital Token Generation

After completing the registration, the system automatically generates a **digital token (Token No: Q145)** for Rahul. Along with the token number, the system displays his current queue position and provides an estimated waiting time of approximately 10–15 minutes. This gives Rahul a clear idea of when he will be served, removing the uncertainty he usually experiences in traditional queues.

### Real-Time Queue Tracking

Instead of standing in a physical line, Rahul chooses to sit comfortably in the waiting area. Using his mobile device, he can track his queue position in real time. The system continuously updates his position as other customers are served. Additionally, a digital display screen inside the bank shows live queue updates, ensuring transparency for all customers.

Meanwhile, another customer, **Sneha**, is also using the system and appreciates how she can monitor her position without physically standing in line. This reduces crowding and creates a more organized environment inside the bank.

### Notification and Alert System

As Rahul's turn approaches, the system sends him an automated SMS notification:  
*"Dear Rahul, your turn is approaching. Please be ready to proceed to the assigned counter."*

This feature ensures that Rahul does not miss his turn and allows him to utilize his waiting time productively.

### **Service Counter Management**

At the service counters, **Amit**, a bank service agent, uses the admin interface to manage the queue. He calls the next customer by clicking a button, and the corresponding token number is displayed on the screen along with the assigned counter number.

At the same time, the bank manager, **Mrs. Kavita Sharma**, monitors the entire queue system through an administrative dashboard. She can view real-time data such as the number of customers waiting, active service counters, and average service time. Observing a slight increase in customer flow, she quickly assigns an additional service counter to reduce waiting time and maintain efficiency.

### **Service Completion Process**

When Rahul's token number is displayed on the screen, he proceeds directly to the assigned counter. Amit efficiently handles Rahul's request and completes the transaction within a few minutes. Once the service is completed, the system automatically updates Rahul's token status to **"Completed."**

Immediately after, the system calls the next customer in the queue, ensuring a smooth and continuous flow of operations without any confusion or delay.

### **Analytics and Performance Tracking**

Behind the scenes, the system records important data such as total customers served, average waiting time, and service duration. Mrs. Kavita Sharma later analyzes this data using the analytics dashboard to identify peak hours, evaluate staff performance, and make informed decisions to further improve service quality.

### **Training and System Support**

The bank staff, including Amit, had earlier received training on how to use the Digital Queue Management System effectively. The system also provides support resources such as user guides and troubleshooting assistance, ensuring smooth operation without technical difficulties.

### **Rahul's Experience**

With the Digital Queue Management System, Rahul experiences a significant improvement compared to traditional queue systems. He no longer has to stand in long lines or worry about missing his turn. The ability to track his position in real time and receive timely notifications allows him to manage his time efficiently.

## Conclusion of the Case Study

The Digital Queue Management System transforms the conventional queue handling process into a smart, efficient, and user-friendly experience. It benefits customers like Rahul by reducing waiting time, minimizing frustration, and providing transparency. At the same time, it helps organizations improve operational efficiency, optimize resource utilization, and deliver better customer service.

This case study clearly demonstrates how the implementation of a digital solution can significantly enhance both customer satisfaction and organizational performance.

## SYSTEM REQUIREMENTS

### Software Requirements

To ensure the efficient development, deployment, and operation of the **Digital Queue Management System**, it is essential to have an appropriate software environment. The following software components are required to support the functionality, scalability, and security of the system.

#### 1. Software Requirements

These tools and technologies are necessary for designing, developing, testing, and executing the application.

**.QR Code Generator Library (e.g., qrcode npm package):** Used to generate QR codes for quick user access to the system.

- **Operating System:**

Windows 10/11, macOS, or Linux-based systems are supported, enabling cross-platform development and deployment.

- **Node.js (Version 16 or above):**

Provides a robust runtime environment for executing server-side JavaScript and handling asynchronous operations efficiently.

- **npm (Node Package Manager) (Version 8 or above):**

Facilitates the installation and management of project dependencies and external libraries.

- **React.js:**

A widely used JavaScript library for building responsive and dynamic user interfaces, ensuring a seamless user experience.

- **Web Browser:**

Modern browsers such as Google Chrome, Mozilla Firefox, or Microsoft Edge (latest versions) are required for testing and running the application.

- **Express.js:**

A lightweight and flexible web application framework for Node.js, used to develop RESTful APIs and manage backend routing.

- **MongoDB:**

A NoSQL database system utilized for storing and managing application data, including user information, queue tokens, and analytics.

- **Mongoose:**

An Object Data Modeling (ODM) library that provides a structured schema-based solution to interact with MongoDB.

- **JSON Web Token (JWT):**

Used for implementing secure authentication mechanisms and role-based access control within the system.

- **Bcrypt:**

A cryptographic library used for hashing user passwords, ensuring data security and protection against unauthorized access.

- **API Testing Tools (Postman / Thunder Client):**

Used for testing, validating, and debugging API endpoints during the development process.

- **Integrated Development Environment (IDE):**

Visual Studio Code is recommended due to its extensive support for extensions, debugging, and version control integration.

- **Version Control System:**

Git, along with GitHub, is used for source code management, collaboration, and maintaining version history.

## **Hardware Requirements**

The following hardware specifications are required to ensure smooth development and execution of the Digital Queue Management System.

- **Processor:**

Minimum: Intel Core i3 or equivalent

Recommended: Intel Core i5 or higher for improved performance and multitasking capabilities.

- **Random Access Memory (RAM):**

Minimum: 4 GB

Recommended: 8 GB or higher to efficiently run development tools, database services, and browsers concurrently.

- **Storage:**

Minimum of 1 GB free disk space is required for project setup and dependencies. Solid State Drive (SSD) is recommended for faster data access and improved system performance.

- **Display:**

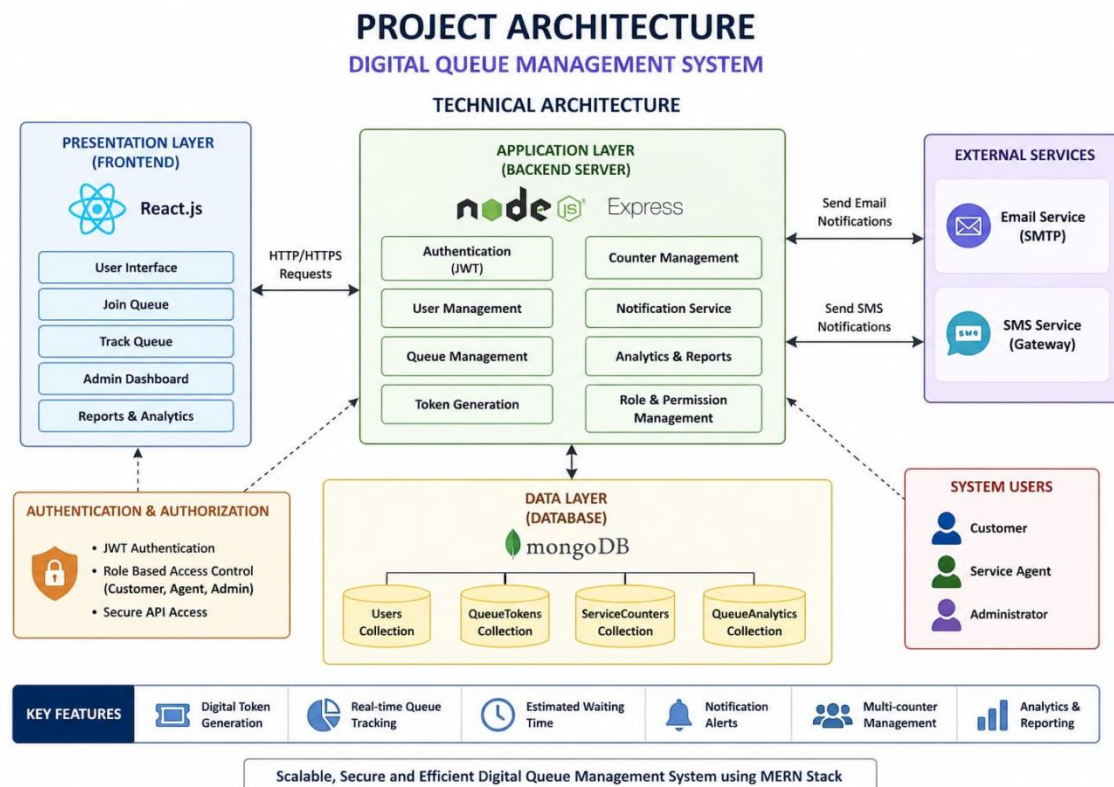
A screen resolution of at least 1366 × 768 pixels is recommended for effective development and interface visualization.

- **Network Requirements:**

A stable internet connection is necessary for downloading dependencies, accessing cloud services, API integration, and real-time updates.

## PROJECT ARCHITECTURE

### Technical Architecture:



In this architecture, the flow begins with a user (represented by the **“User” node**) interacting with the Digital Queue Management System frontend (represented by the **“Frontend – React.js” node**). The user accesses the system through a web browser or mobile device to perform actions such as joining a queue, checking queue status, or managing queue operations.

A QR code module is integrated as an entry access point, enabling users to scan and directly interact with the frontend application without manual URL entry.

The user submits a request through the frontend interface, such as generating a token or viewing queue position. The frontend then sends this request to the backend server (represented by the **“Backend – Node.js & Express.js” node**) using HTTP/HTTPS protocols.

The backend receives the request and processes it according to the application logic. This includes handling operations such as token generation, queue position calculation, service counter allocation, and notification management. The backend ensures that all business rules are applied correctly before proceeding further.

The backend then communicates with the database (represented by the **“Database – MongoDB” node**) to store and retrieve data. The database securely maintains all system-related information, including user details, queue tokens, service counter data, and queue analytics. This ensures data consistency and efficient retrieval during real-time operations.

Additionally, the backend integrates with external services (represented by the **“External Services” node**) such as SMS gateways and email servers. These services are used to send notifications and alerts to users when their turn is approaching or when important updates occur in the queue.

Once the backend completes processing the request, it sends an appropriate response back to the frontend. The frontend then updates the user interface dynamically and displays the latest information, such as updated queue position, estimated waiting time, or confirmation messages.

This continuous interaction between the frontend, backend, database, and external services ensures a seamless and real-time queue management experience for users.

The “style” elements in the architecture diagram (if included) are optional and are used to visually differentiate components such as users, services, and storage systems for better clarity and presentation.

The Digital Queue Management System follows a **full-stack architecture using the MERN stack**.

## **Frontend Layer**

The frontend is developed using **React.js**.

This interface allows customers and administrators to interact with the system.

Frontend capabilities include:



- Joining queues
- Viewing queue position
- Tracking waiting time
- Managing queues

## Backend Layer

The backend is developed using **Node.js and Express.js**.

The backend handles the application logic and API requests.

Backend responsibilities include:

- Token generation
- Queue position management
- Notification handling
- Service counter management

## Database Layer

The system uses **MongoDB** as the database.

MongoDB stores information such as:

- Customer queue tokens
- Service counter details
- Queue status information
- Queue analytics data

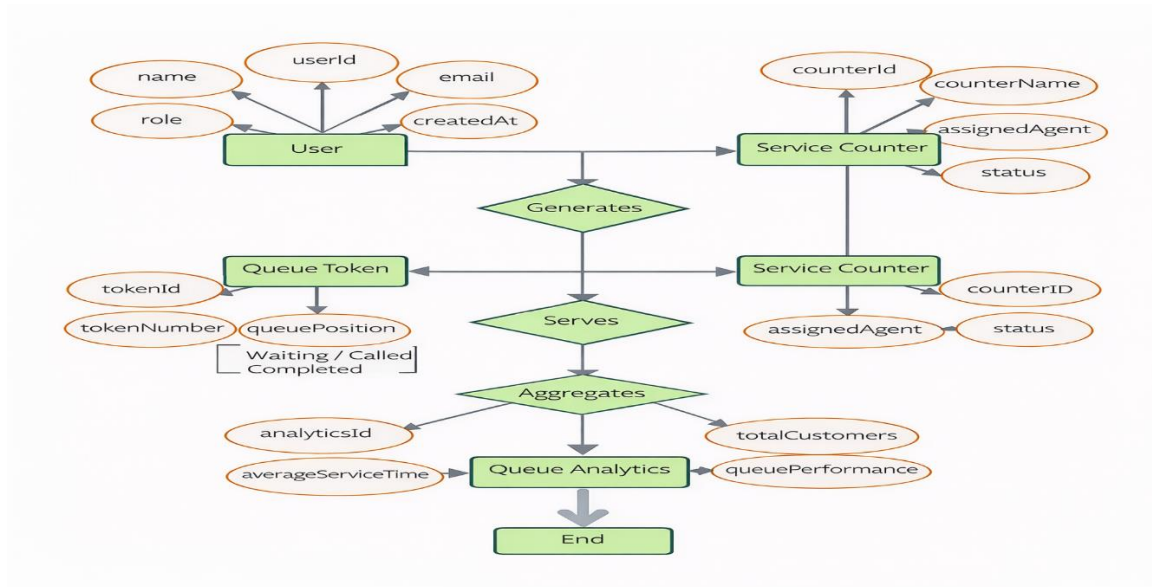
## Authentication Layer

The system uses **JWT Authentication** for secure login and role-based access.

Roles include:

- Customers
- Service Agents
- Administrators

## ER Diagram (Entity Relationship)



## ENTITY-RELATIONSHIP (ER) MODEL

The **Digital Queue Management System** utilizes an Entity-Relationship (ER) model to represent the logical structure of the database and the relationships between different entities within the system. This model plays a crucial role in ensuring data consistency, efficient queue handling, and seamless interaction between various system components.

The ER model defines key entities such as users, queue tokens, service counters, and analytics data, along with their attributes and relationships.

### Main Entities in the System

#### 1. User Entity

The User entity stores information related to all individuals interacting with the system, including customers, service agents, and administrators. This entity is fundamental for authentication and role-based access control.

##### Attributes:

- userId (Primary Key)
- name
- email
- password
- role (Customer / Service Agent / Administrator)
- createdAt

A user can perform different operations depending on their role, such as generating queue tokens, managing queues, or monitoring system performance.

## **2. Queue Token Entity**

The Queue Token entity represents the digital tokens generated when customers join the queue. It is essential for tracking the order and status of customers in the queue.

### **Attributes:**

- tokenId (Primary Key)
- userId (Foreign Key)
- tokenNumber
- queuePosition
- status (Waiting / Called / Completed)

Each queue token is uniquely associated with a specific user and is used to manage queue flow efficiently.

## **3. Service Counter Entity**

The Service Counter entity represents the service points where customers are attended. It helps in managing multiple counters and distributing workload among service agents.

### **Attributes:**

- counterId (Primary Key)
- counterName
- assignedAgent (Foreign Key → User)
- status (Active / Inactive)

Each service counter is managed by a service agent and can handle multiple queue tokens over time.

## **4. Queue Analytics Entity**

The Queue Analytics entity stores statistical and performance-related data that helps administrators evaluate system efficiency and optimize operations.

### **Attributes:**

- analyticsId (Primary Key)
- totalCustomers
- averageServiceTime

- queuePerformance

This entity aggregates data from queue tokens and service counters to generate meaningful insights.

## Relationships Between Entities

The relationships between the entities define how data is connected and interact within the system:

- A **User** can generate multiple **Queue Tokens** (One-to-Many relationship).
- Each **Queue Token** is associated with exactly one **User** (Many-to-One relationship).
- A **Service Counter** serves multiple **Queue Tokens** (One-to-Many relationship).
- A **Service Agent (User)** can be assigned to one or more **Service Counters** (One-to-Many relationship).
- The **Queue Analytics** entity aggregates data from multiple **Queue Tokens** and **Service Counters** to provide performance insights.

## KEY FEATURES

The **Digital Queue Management System** provides a set of advanced features designed to improve queue handling, enhance user experience, and optimize service efficiency.

### 1. Digital Token Generation

The system allows customers to generate queue tokens digitally through a web interface or QR code. Each token is assigned a unique number and queue position automatically, eliminating the need for physical tickets.

### 2. Real-Time Queue Tracking

Users can monitor their queue position in real time. The system continuously updates queue status, ensuring transparency and reducing uncertainty for customers.

### 3. Estimated Waiting Time

The system calculates and displays an approximate waiting time based on queue length and service speed, helping users plan their time effectively.

### 4. Notification Alerts

Automated notifications are sent to users via SMS or email when their turn is approaching. This reduces the need for customers to wait physically in the queue.

### 5. Multi-Counter Management

The system supports multiple service counters and efficiently distributes customers among them. Administrators can assign or reassign counters dynamically based on demand.

## **6. Admin Dashboard**

A centralized dashboard is provided for administrators to monitor queue activity, manage service counters, and oversee overall system operations in real time.

## **7. Role-Based Access Control**

The system implements secure authentication using role-based access, allowing different functionalities for customers, service agents, and administrators.

## **8. Queue Analytics and Reporting**

The system collects and analyzes data such as total customers served, average waiting time, and service performance. These insights help improve operational efficiency.

## **9. User-Friendly Interface**

The application provides a clean and intuitive interface, making it easy for users of all technical backgrounds to interact with the system.

## **10. Scalable and Flexible Architecture**

Built using the MERN stack, the system is highly scalable and can be adapted for various domains such as banks, hospitals, and government offices.

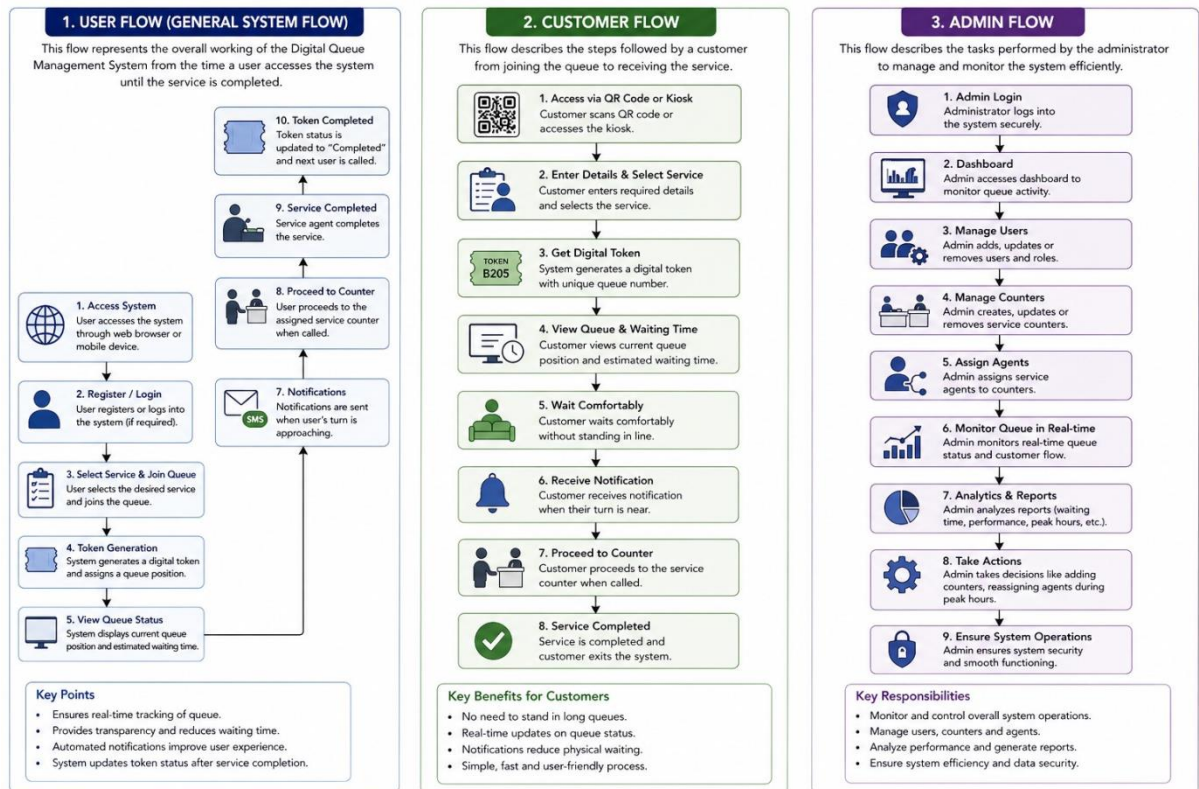
The Digital Queue Management System defines clear workflows for different types of users to ensure smooth and efficient operation. These flows describe how users interact with the system and how processes are executed step by step.

## **11. QR Code-Based Access**

Users can scan a QR code placed at the service location to instantly access the queue system, ensuring quick and contactless registration.

## SYSTEM FLOW (USER FLOW, ADMIN FLOW, CUSTOMER FLOW)

The Digital Queue Management System defines clear workflows for different types of users to ensure smooth and efficient operation. These flows describe how users interact with the system and how processes are executed step by step.



### 1. User Flow (General System Flow)

The user flow represents the overall working of the system from entry to service completion.

#### Steps:

1. User accesses the system through a web browser or mobile device.
2. User registers or logs into the system (if required).
3. User selects the desired service and joins the queue.
4. The system generates a digital token and assigns a queue position.
5. The system calculates and displays the estimated waiting time.
6. Queue status is updated continuously in real time.

7. Notifications are sent when the user's turn is approaching.
8. User proceeds to the assigned service counter.
9. Service agent completes the service.
10. Token status is updated to "Completed," and the next user is called.

## **2. Customer Flow**

The customer flow specifically focuses on how customers interact with the system to receive services.

### **Steps:**

1. Customer accesses the system via QR code or kiosk.
2. Customer enters required details and selects the service.
3. System generates a digital token with queue number.
4. Customer views current queue position and waiting time.
5. Customer waits comfortably without standing in line.
6. Customer receives notification when their turn is near.
7. Customer proceeds to the service counter when called.
8. Service is completed, and the customer exits the system.

## **3. Admin Flow**

The admin flow describes how administrators manage and control the system operations.

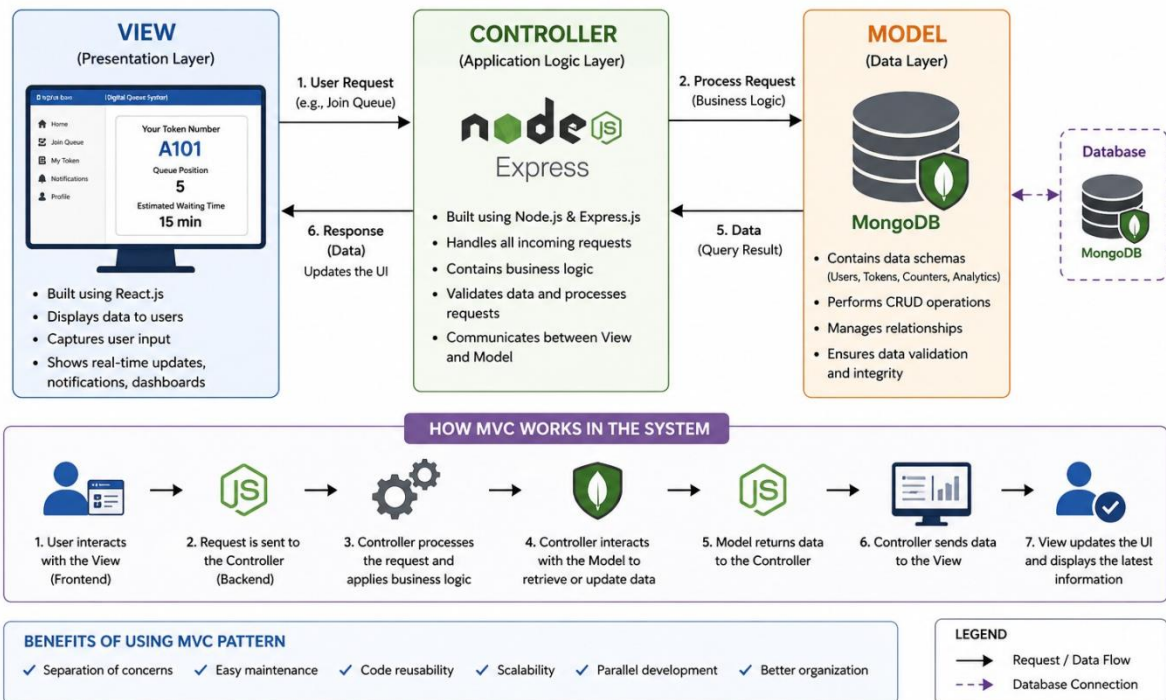
### **Steps:**

1. Administrator logs into the system securely.
2. Admin accesses the dashboard to monitor queue activity.
3. Admin manages users (add/update/remove roles).
4. Admin creates and manages service counters.
5. Admin assigns service agents to counters.
6. Admin monitors real-time queue status and customer flow.
7. Admin analyzes reports such as waiting time and service performance.
8. Admin takes decisions such as adding counters during peak hours.
9. Admin ensures system security and smooth functioning.

## MVC PATTERN (Model–View–Controller Architecture)

### MVC PATTERN – DIGITAL QUEUE MANAGEMENT SYSTEM

The system is designed using MVC (Model – View – Controller) architecture to separate concerns and improve maintainability.



The **Digital Queue Management System** follows the Model–View–Controller (MVC) architectural pattern, which helps in organizing the application into separate layers. This separation of concerns improves code maintainability, scalability, and development efficiency.



The MVC pattern divides the application into three main components: Model, View, and Controller.

## **1. Model (Data Layer)**

The Model represents the data structure and business logic of the application. It is responsible for managing data, interacting with the database, and enforcing rules related to data processing.

In this system, the Model is implemented using MongoDB along with Mongoose schemas.

### **Responsibilities of the Model include:**

- Defining data schemas (Users, Queue Tokens, Service Counters, Analytics)
- Performing CRUD operations (Create, Read, Update, Delete)
- Managing relationships between data entities
- Ensuring data validation and integrity

## **2. View (Presentation Layer)**

The View represents the user interface of the application. It is responsible for displaying data to users and capturing user interactions.

In this system, the View is developed using React.js, which provides a dynamic and responsive interface.

### **Responsibilities of the View include:**

- Displaying queue information (token number, position, waiting time)
- Providing forms for user input (registration, service selection)
- Showing real-time updates and notifications
- Rendering admin dashboards and reports
- The View also displays QR codes for quick system access and user onboarding.

## **3. Controller (Application Logic Layer)**

The Controller acts as an intermediary between the Model and the View. It processes user requests, applies business logic, and determines how data should be handled.

In this system, the Controller is implemented using Node.js and Express.js.

### **Responsibilities of the Controller include:**

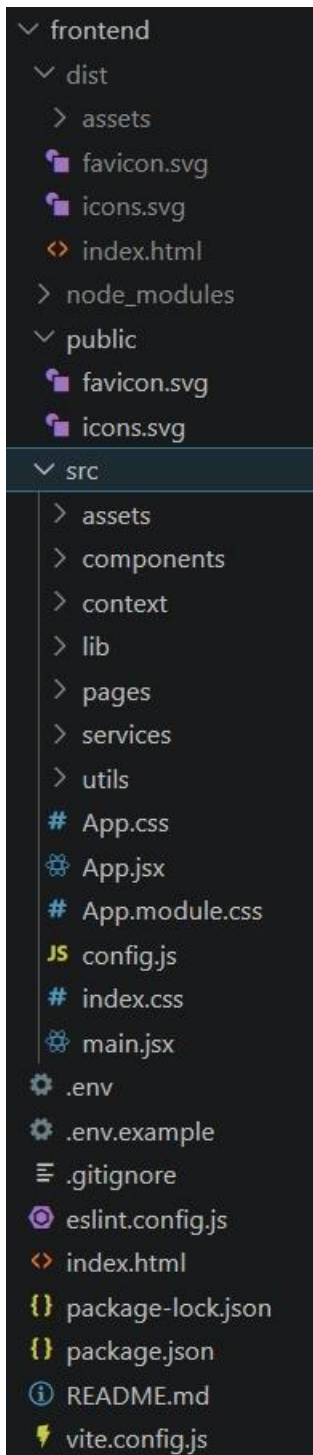
- Handling incoming API requests from the frontend
- Processing user actions (join queue, generate token, update status)
- Communicating with the Model to retrieve or update data
- Sending appropriate responses back to the View
- Managing authentication and authorization

### **Creating project folder**

- Create a new folder with your .
- Inside that folder create two new folders.
- Name one as client.
- Name another one as server.
- Now open that folder in VS Code.

### **● Client setup (installing react app)**

- Open the frontend folder in the terminal of VSCode.



- `npm create vite@latest . -- --template react`
- Select React framework from the given options.
- Select a framework:
  - | React
- Select JavaScript variant from the given options.
- Select a variant:
  - | JavaScript

- Now lets navigate to the frontend folder by giving the following command.

- `cd frontend`

- To install all the packages run the following command.

- `npm install`

- To start the React server type the following command.

- `npm run dev`

- **Server setup (npm init)**

- Open backend folder in terminal of VSCode.



- npm init -y
- Create files:
  - server.js

- Create folders:

- models
- controllers
- routes
- middleware
- services

## **BACKEND DEVELOPMENT:**

### **Backend Structure:**

#### **Controllers**

- adminController.js → Handles all admin-related operations such as managing users, system settings, and administrative tasks.
- authController.js → Handles user authentication such as login, registration, and token generation.
- queueController.js → Manages queue-related operations such as token generation, queue updates, and tracking.
- staffController.js → Handles staff-related operations and management.

#### **Middleware**

- authMiddleware.js → Middleware for authentication and authorization using tokens (JWT validation).

#### **Models**

- Counter.js → Schema/model for maintaining counters used in the system.
- Feedback.js → Schema/model for storing user feedback.
- QueueLog.js → Schema/model for tracking queue history and logs.
- Sequence.js → Schema/model for generating unique sequence values.
- Token.js → Schema/model for managing tokens in the queue system.
- User.js → Schema/model for user accounts and details.

## Routes

- adminRoutes.js → Defines API routes for admin operations.
- authRoutes.js → Defines API routes for authentication (login/register).
- queueRoutes.js → Defines API routes for queue management.
- staffRoutes.js → Defines API routes for staff-related operations.

## Services

- emailService.js → Handles email sending functionality.
- queueService.js → Contains business logic for queue processing.

Handles QR code generation for system access links.

.

## DATABASE DEVELOPMENT:

### 1. Configure MongoDB:

- Install Mongoose.

In your Node.js project directory, run:

```
npm install mongoose
```

- Create database connection.
- Create Schemas & Models.

### Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below.

```
full stack pro > backend > ⚙️ .env.example
1  MONGO_URI=mongodb://127.0.0.1:27017/queue_management
2  JWT_SECRET=supersecret
3  PORT=5000
4  CLIENT_URL=http://localhost:5173
5
6  TOKEN_RESPONSE_MINUTES=5
7  DEFAULT_AVG_SERVICE_MINUTES=6
8  TURN_APPROACHING_THRESHOLD=1
9
10 EMAILJS_ENABLED=false
11 EMAILJS_PUBLIC_KEY=2_jx7fcUMJZXTsUT9
12 EMAILJS_SERVICE_ID=service_0tyi4th
13 EMAILJS_TEMPLATE_ID=template_jatma2k
14 EMAILJS_TO_EMAIL=hemachandralanka444@gmail.com
15 EMAILJS_FROM_NAME=Digital Queue Management
16
```

## Create Schemas:

Firstly, configure the Schemas for MongoDB database, to store the data in such a pattern. The Schemas for this application look alike to the one provided below.

### 1.User.js → Schema/model for user accounts.

```
full stack pro > backend > models > JS User.js > ...
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcryptjs');
3
4  const UserSchema = new mongoose.Schema({
5    name: { type: String, required: true },
6    email: { type: String, required: true, unique: true },
7    password: { type: String, required: true },
8    role: { type: String, enum: ['customer', 'agent', 'admin'], default: 'customer' },
9    assignedCounter: { type: mongoose.Schema.Types.ObjectId, ref: 'Counter' }
10 }, { timestamps: true });
11
12 UserSchema.pre('save', async function() {
13   if (!this.isModified('password')) return;
14   const salt = await bcrypt.genSalt(10);
15   this.password = await bcrypt.hash(this.password, salt);
16 });
17
18 UserSchema.methods.matchPassword = (parameter) enteredPassword: any }) {
19   return await bcrypt.compare(enteredPassword, this.password);
20 };
21
22 module.exports = mongoose.model('User', UserSchema);
23
```



## 2.Token.js → Schema/model for managing tokens in the system.

```
full stack pro > backend > models > JS Token.js > ...
1  const mongoose = require('mongoose');
2
3  const NotificationSchema = new mongoose.Schema({
4    type: { type: String, required: true },
5    message: { type: String, required: true },
6    createdAt: { type: Date, default: Date.now },
7  }, { _id: false });
8
9  const TokenSchema = new mongoose.Schema({
10   tokenNumber: { type: String, required: true, unique: true },
11   customerId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
12   counterId: { type: mongoose.Schema.Types.ObjectId, ref: 'Counter', default: null },
13   // Bank service type – drives token prefix and counter routing
14   serviceType: {
15     type: String,
16     enum: ['deposit', 'withdrawal', 'loan', 'account_opening', 'general_enquiry'],
17     default: 'general_enquiry',
18   },
19   priorityType: { type: String, enum: ['normal', 'senior', 'emergency'], default: 'normal' },
20   // Full lifecycle: waiting → assigned → active → completed | expired
21   status: {
22     type: String,
23     enum: ['waiting', 'assigned', 'active', 'completed', 'expired'],
24     default: 'waiting',
25   },
26   assignedAt: { type: Date, default: null },
27   acceptedAt: { type: Date, default: null },
28   completedAt: { type: Date, default: null },
29   serviceStartTime: { type: Date, default: null },
30   serviceEndTime: { type: Date, default: null },
31   serviceDuration: { type: Number, default: null },
32   serviceDurationMinutes: { type: Number, default: null },
33   estimatedWaitMinutes: { type: Number, default: 0 },
34   lastKnownPosition: { type: Number, default: null },
35   feedbackSubmitted: { type: Boolean, default: false },
36   notificationHistory: { type: [NotificationSchema], default: [] },
37 }, { timestamps: true });
```

```
37 }, { timestamps: true });
38
39 module.exports = mongoose.model('Token', TokenSchema);
40
```

## 3. QueueLog.js → Schema/model for storing queue logs.

```

full stack pro > backend > models > JS QueueLog.js > ...
1  const mongoose = require('mongoose');
2
3  const QueueLogSchema = new mongoose.Schema({
4    tokenId: { type: mongoose.Schema.Types.ObjectId, ref: 'Token', required: true },
5    customerId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
6    counterId: { type: mongoose.Schema.Types.ObjectId, ref: 'Counter', default: null },
7    agentId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', default: null },
8    startTime: { type: Date },
9    endTime: { type: Date },
10   serviceDuration: { type: Number } // duration in minutes
11  }, { timestamps: true });
12
13  module.exports = mongoose.model('QueueLog', QueueLogSchema);
14

```

#### 4 . Feedback.js → Schema/model for storing user feedback.

```

full stack pro > backend > models > JS Feedback.js > ...
1  const mongoose = require('mongoose');
2
3  const FeedbackSchema = new mongoose.Schema({
4    customerId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
5    tokenId: { type: mongoose.Schema.Types.ObjectId, ref: 'Token', required: true },
6    agentId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', default: null },
7    counterId: { type: mongoose.Schema.Types.ObjectId, ref: 'Counter', default: null },
8    rating: { type: Number, required: true, min: 1, max: 5 },
9    comment: { type: String },
10   createdAt: { type: Date, default: Date.now }
11  });
12
13  FeedbackSchema.index({ tokenId: 1 }, { unique: true });
14
15  module.exports = mongoose.model('Feedback', FeedbackSchema);
16

```

#### 5 . Counter.js → Schema/model for maintaining counters.

```

full stack pro > backend > models > JS Counter.js > ...
1  const mongoose = require('mongoose');
2
3  const CounterSchema = new mongoose.Schema({
4    counterName: { type: String, required: true, unique: true },
5    assignedAgent: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
6    activeToken: { type: mongoose.Schema.Types.ObjectId, ref: 'Token', default: null },
7    status: { type: String, enum: ['active', 'inactive'], default: 'inactive' },
8  }, { timestamps: true });
9
10  module.exports = mongoose.model('Counter', CounterSchema);
11

```

## 6 .Sequence.js → Schema/model for generating unique sequence values.

```
full stack pro > backend > models > JS Sequence.js > ...
1  const mongoose = require('mongoose');
2
3  // Atomic sequence counter per service type per day.
4  // _id format: "deposit:2026-04-26", "withdrawal:2026-04-26", etc.
5  const SequenceSchema = new mongoose.Schema({
6    _id: { type: String, required: true },
7    seq: { type: Number, default: 0 },
8  });
9
10 module.exports = mongoose.model('Sequence', SequenceSchema);
11
```

## FRONT-END DEVELOPMENT:

### Frontend Structure:

#### Components (src/components)

- Reusable UI components used across the application.

#### Pages (src/pages)

- Contains different pages/screens of the application.

#### Services (src/services)

- Handles API calls and business logic for frontend.

#### Context (src/context)

- Manages global state using React Context API.

#### Utils (src/utils)

- Contains utility/helper functions.

#### Assets (src/assets)

- Contains static files such as images and icons.

### Main Files

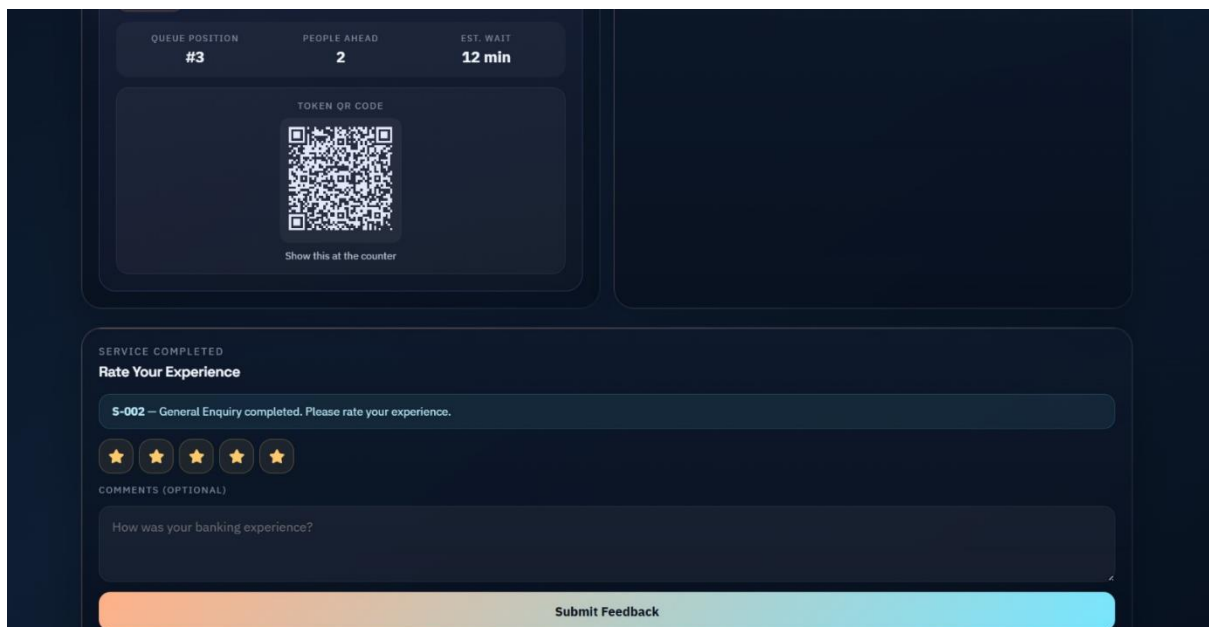
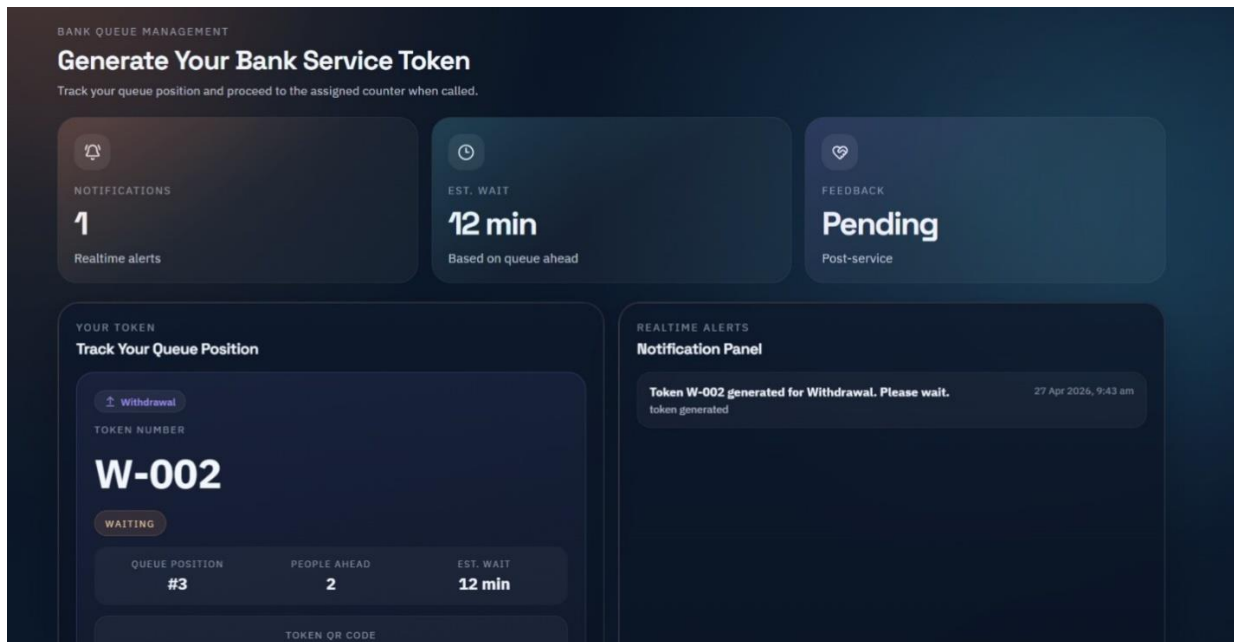
- App.jsx → Main component that handles routing and renders different pages.
- main.jsx → Entry point of the React application.
- App.css → Contains styles for the application.
- App.module.css → Contains modular CSS styles.
- config.js → Contains configuration details.
- index.css → Global styling for the application.

## Output ScreenShots:

### Landing Page:

The screenshot displays a dark-themed landing page for a 'DIGITAL QUEUE MANAGEMENT' system. At the top, the title 'Sign in to your workspace' is prominently displayed, followed by the instruction 'Select your role to continue'. Below this, three role-based login cards are presented: 'Admin' (with a shield icon and description: 'Manage queue routing, counters and analytics.'), 'Customer' (with a person icon and description: 'Generate token, track position and submit feedback.'), and 'Counter Staff' (with a monitor icon and description: 'Accept tokens, set service duration and complete service.'). A section titled 'Signing in as Admin' follows, containing input fields for 'EMAIL' and 'PASSWORD', each with a placeholder text 'Enter your email' and 'Enter your password' respectively. A large, gradient-colored 'Sign in' button with a lock icon is positioned at the bottom of the form.

### Register.jsx(Admin):



**Login.jsx:**

DIGITAL QUEUE MANAGEMENT

Sign in to your workspace

Select your role to continue

Admin

Manage queue routing, counters and analytics.

Customer

Generate token, track position and submit feedback.

Counter Staff

Accept tokens, set service duration and complete service.

Signing in as Admin

EMAIL

Enter your email

PASSWORD

Enter your password

Sign in

## Admin HomePage:

Queue Overview

SERVED TODAY

1

Completed tokens

WAITING

3

In queue

BEST COUNTER

Loan Counter

1 served

AVG RATING

—

Customer feedback

3 IN QUEUE

Waiting Tokens

G-001

Customer Normal

NORMAL

Assign to counter

W-001

karthik

NORMAL

Assign to counter

W-002

jaya sri

NORMAL

Assign to counter

0 ROUTED

Assigned Tokens

No assigned tokens.

0 BEING SERVED

Active Tokens

No active tokens.

COUNTER MANAGEMENT

OPERATIONS

COUNTER MANAGEMENT

Add Counter

COUNTER NAME

e.g. Counter Delta

+ Add counter

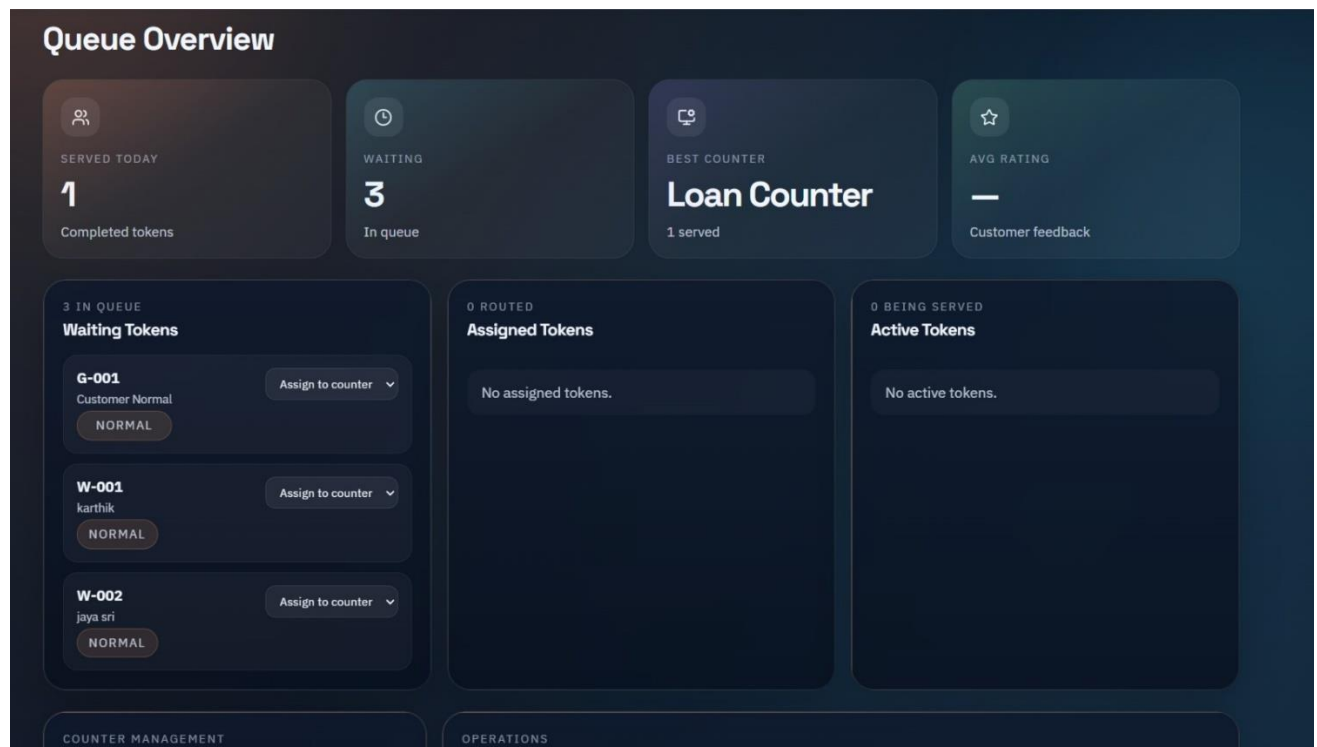
OPERATIONS

Counters & Staff

| COUNTER                 | STATUS   | ASSIGNED STAFF | ACTION     |
|-------------------------|----------|----------------|------------|
| Counter Alpha           | ACTIVE   | Unassigned     | Deactivate |
| counter Beta            | INACTIVE | Unassigned     | Activate   |
| Counter Beta            | INACTIVE | Unassigned     | Activate   |
| Counter Gamma           | INACTIVE | Unassigned     | Activate   |
| Deposit Counter         | INACTIVE | Unassigned     | Activate   |
| Withdrawal Counter      | INACTIVE | Unassigned     | Activate   |
| Loan Counter            | ACTIVE   | Agent Debug    | Deactivate |
| Account Opening Counter | INACTIVE | Unassigned     | Activate   |
| General Enquiry Counter | INACTIVE | Unassigned     | Activate   |



User's Page:



**My Token Page:**



↓ Deposit

TOKEN NUMBER

**D-005**

WAITING

QUEUE POSITION

**#4**

PEOPLE AHEAD

**3**

EST. WAIT

**18 min**

TOKEN QR CODE



Show this at the counter

## User's Login Page:

Digital Queue

Google Password Manager

Ask Gemini

localhost:5173/login

Queue Nexus

Register

DIGITAL QUEUE MANAGEMENT

### Sign in to your workspace

Select your role to continue

**Admin**  
Manage queue routing, counters and analytics.

**Customer**  
Generate token, track position and submit feedback.

**Counter Staff**  
Accept tokens, set service duration and complete service.

Signing in as **Customer**

EMAIL

chandre@gmail.com

PASSWORD

.....

Sign In

## User's Home Page:

Digital Queue

Google Password Manager

Ask Gemini

localhost:5173/customer

Queue Nexus

LIVE OPERATIONS

hema CUSTOMER

Sign out

BANK QUEUE MANAGEMENT

### Generate Your Bank Service Token

Track your queue position and proceed to the assigned counter when called.

NOTIFICATIONS

7

Realtime alerts

EST. WAIT

18 min

Based on queue ahead

FEEDBACK

Pending

Post-service

YOUR TOKEN

Track Your Queue Position

Deposit

TOKEN NUMBER

D-005

WAITING

| QUEUE POSITION | PEOPLE AHEAD | EST. WAIT |
|----------------|--------------|-----------|
| #4             | 3            | 18 min    |

REALTIME ALERTS

Notification Panel

- Token D-005 generated for Deposit. Please wait. token generated 2 May 2026, 5:40 pm
- Token D-005 generated for Deposit. Please wait. token generated 2 May 2026, 5:40 pm
- Please rate your Deposit experience. feedback requested 1 May 2026, 10:11 am
- Service completed at Deposit Counter. Thank you for banking with us. 1 May 2026, 10:11 am

## Token's Tracking Page:

↓ Deposit

TOKEN NUMBER

**D-005**

WAITING

QUEUE POSITION

**#4**

PEOPLE AHEAD

**3**

EST. WAIT

**18 min**

TOKEN QR CODE



Show this at the counter

## Project Video:

[https://drive.google.com/file/d/1HU6ivyCSSVfCBmkqdeptUFr\\_HScXsikQ/view?usp=drivesdk](https://drive.google.com/file/d/1HU6ivyCSSVfCBmkqdeptUFr_HScXsikQ/view?usp=drivesdk)

## Frontend Explanation Link:

<https://drive.google.com/file/d/16E2zzcIte97u4Tf4qnVKzoEGI3Jgmksu/view?usp=drivesdk>

**Backend Explanation Link:**

**<https://drive.google.com/file/d/18XAFfq9OLOG7ZE5qGbdpSeLW7mfTHDRv/view?usp=drivesdk>**

**Code Drive Link :**

**[https://drive.google.com/drive/u/0/folders/1nhM\\_O9dp44c0SW5mrhZtAOyxW\\_hFxocj](https://drive.google.com/drive/u/0/folders/1nhM_O9dp44c0SW5mrhZtAOyxW_hFxocj)**

**Github Link:**

**[https://github.com/dheeraj-reddy8/digital\\_queue\\_mangement](https://github.com/dheeraj-reddy8/digital_queue_mangement)**